Recommending Games Based on Preference and Description: SteamRec

Ki Hong Park
Computer Science
Virginia Tech
Alexandria, VA, USA
stevep00@vt.edu

Christopher Williams
Computer Science
Virginia Tech
Alexandria, VA, USA
cdw24@vt.edu

***Abstract*—Steam's native recommendation engine surfaces games based on play history, social connections, and popularity — leaving users with limited tools to discover new titles unrelated to play history or social network. This paper presents SteamRec, a natural language game recommendation system. Users provide a description of the type of game they want to play, which is passed to ChatGPT via the OpenAI API alongside structured game metadata. The LLM returns a ranked list of matching titles, which are then resolved against a MongoDB database populated via the Steam Store API. The system returns enriched results for each recommendation, including the game name, Steam Store link, current price, community tags, review scores, and related titles. The full stack consists of a React/TypeScript frontend, a MongoDB NoSQL backend, and a Dockerized deployment environment. SteamRec demonstrates how combining semantic reasoning from large language models with structured database retrieval can surface relevant, non-obvious game recommendations beyond what traditional filtering approaches provide.**

I. Introduction

The video game industry has experienced explosive growth over the past decade, with Steam burgeoning as the leader in the video game marketplace, hosting tens of thousands of titles that span across nearly every genre, style, and price. For players, the sheer volume of titles is as much a benefit as a challenge: discovering new games outside of ones own interests and searching for specific moods, mechanics, or narratives is difficult. Steam's built-in recommendation system attempts to address this by analyzing user play hstory, behavior patterns, and trends of users across the platform, but this approach is reactive [5].

This limitation is exacerbated for users seeking games outside their established history. A player that wants "a short, atmosphereic horror game with minimal combat" or "a co-op puzzle game with a good story." These preferences expressed in natural language are normal in conversation but not easily traditional filtering systems, which rely on structured interaction data rather than open-ended natural language input.

Recent advances in large language models (LLMs) have opened new possibilities for bridging this gap. LLMs are capable of interpreting natural language queries, reasoning over item descriptions, and returning relevant results without requiring users to conform to search syntax or pre-defined categories that force the loss of specificity. When combined with a structured database of game metadata, this capability can power a recommendation system that is both expressive and precise.

We present the proposal for SteamRec, a system that enables usersto describe the kind of game they are looking for in natural language and receive relevant, enriched recommendations drawn from a MongoDB database populated via the Steam Store API. The system passes the user's query to an OpenAI LLM, which reasons over structured game descriptions to return a list of matching titles. Those titles are then resolved against the database to surface complete metadata: name, price, community tags, review scores, related titles, and a direct Steam Store link.

The remainder of this paper is structured as follows. Section 2 reviews related work in the game recommendation systems and LLM-based recommender architectures. Section 3 describes proposed approaches. Section 4 presents the system design of the project. We conclude with our sources and appendix which includes task assignments to each team member.

II. Related Work

A. Game Recommendation on Steam

Several prior works have examined the challenge of game recommendation specifically within the Steam ecosystem. Batra et al. (2023) built a recommendation system using Apache Spark and implicit/explicit user signals to return a ranked list of titles a user is likely to enjoy [1]. While effective for users with established play histories, this approach inherits the core limitation of collaborative filtering: it cannot serve users seeking games outside their prior preferences. Blue, Garcia, and Turner (2024) similarly analyzed Steam profiles and reviews to understand how recommendation engines surface titles, noting that smaller, independent game studios are systematically disadvantaged by popularity-driven algorithms [2]. Their work underscores the need for systems that can surface relevant, non-obvious titles, which is a central motivation for SteamRec.

B. Natural Language and AI-Driven Recommendation

The application of large language models to recommendation has grown rapidly in recent years. Wu et al. (2023) provide a comprehensive survey of the space, categorizing model-based recommender systems into two paradigms: those that use language models as feature extractors within traditional

pipelines, and those that treat the model itself as the recommender [6]. SteamRec falls broadly into the latter category, using an OpenAI model to reason over game descriptions and return ranked suggestions, which are then enriched through database retrieval.

One of the most architecturally similar systems to SteamRec is InteRecAgent, introduced by Huang et al. (2023) and published in ACM Transactions on Information Systems. InteRecAgent uses a generative model as the "brain" of a conversational recommender, while traditional recommender models serve as domain-specific tools for retrieval and ranking [4]. The key insight is that language-based reasoning and structured retrieval are complementary rather than competing , an idea that is central to SteamRec's design as well. Where InteRecAgent integrates domain-specific recommender models as callable tools, SteamRec achieves a similar separation by delegating natural language reasoning to the model and precise metadata retrieval to MongoDB.

Friedman et al. (2023) introduced RecLLM, a large-scale conversational recommender for YouTube built on Google's LaMDA model [3]. Their work highlights key engineering challenges in designing language-driven recommender systems, including how to handle a large and evolving item corpus and how to manage multi-turn dialogue while maintaining recommendation relevance. While SteamRec operates in a single-turn query paradigm rather than a multi-turn dialogue, many of the same core challenges, particularly around grounding model outputs in a real item catalog, apply to our system.

Prior work in both Steam-specific recommendation and language model-driven recommender systems reveals a clear gap. Existing Steam recommenders rely heavily on predefined user behavior patterns, while more general model-based systems have not been applied to the gaming domain with a focus on open-ended, description-driven discovery. SteamRec is designed to address this gap directly.

III. Proposed Approaches

A. System Overview

SteamRec is designed as a three-tier system consisting of a React/TypeScript frontend, an OpenAI-powered reasoning layer, and a MongoDB backend populated with game metadata from the Steam Store API. The core interaction flow is as follows:

- User submits natural language description of game
- Query is passed to OpenAI language model with structured response formatting
- Titles are resolved against MongoDB and returns games + metadata and returned to user

The system is containerized with Docker to ensure consistent deployment.

Figure 1 is the general FARM stack that our project is based on. Figure 2 is the general FARM pipeline that our project is based on. The architecture, stack, and pipeline will be altered as necessary during development.

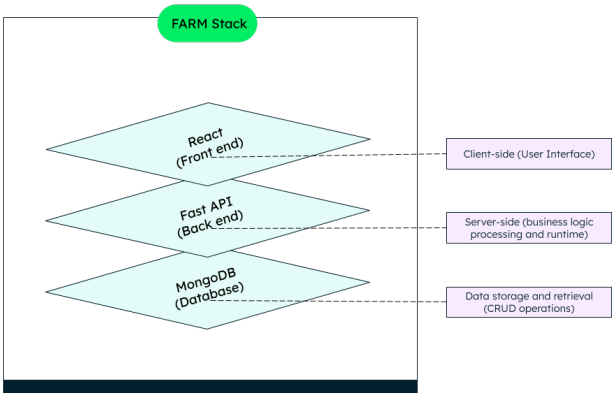

Fig. 1. FARM Stack Layers

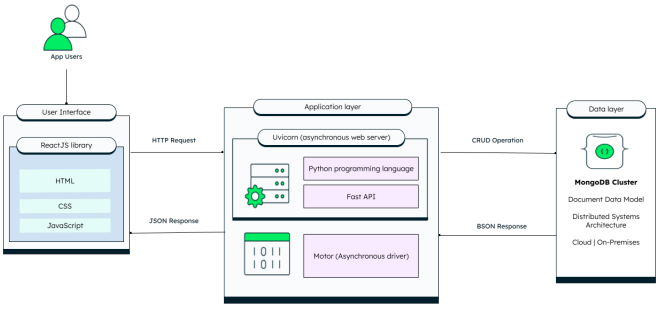

Fig. 2. General FARM Tech Stack Pipeline

B. Data Collection and Database Design

Game metadata will be collected from the Steam Store API, which exposes per-game information. Data will be ingested in batch and persisted in MongoDB. Each document in the database will represent a single game and will include fields for the game's title, store description, community tags, current price, review score, total review count, related game references, and a direct link to the game's Steam Store page.

C. LLM Integration

User queries will be submitted to the OpenAI API alongside a structured prompt containing game descriptions drawn from the database. The prompt is designed to instruct the model to reason over the provided descriptions and return a ranked list of game titles that best match the user's input. This approach keeps the model grounded in the actual contents of the database, reducing the risk of the model hallucinating titles that do not exist in the catalog. The model's output is then used to query MongoDB, retrieving full metadata records for each recommended title.

D. Frontend and UX

The frontend will be implemented in React with TypeScript, providing a single-page interface where users can enter a

natural language game description and receive a list of recommendations. Each result will display the game name, cover image, current price, community tags, review score, related titles, and a direct link to the game's Steam Store page.

E. Containerization and Deployment

All system components (React frontend, backend API server, MongoDB instance) will be containerized using Docker and orchestrated with Docker Compose. Environment variables will be used to manage API keys and database credentials securely across deployment contexts.

IV. System Design

A. Frontend

The frontend of SteamRec will be implemented using React, which will serve as the interactive layer for users to submit steam queries and receive responses based on our backend/LLM integration design.

B. Backend

The backend will be implemented using FastAPI. This was chosen for it's intuitive integration with our chosen database layer and ease of use in combination with React. FastAPI is also considerably lightweight and fast and can support a large number of concurrent persistent connections due to its implementation of non-blocking asynchronous state machine parser.

C. Database

Our choice of database is MongoDB, an intuitive NoSQL Database. MongoDB also supports GenAI data analytics and real-time analysis for the purpose of steam API queries. The combination of FastAPI, React, and MongoDB is a well-known full technology stack known as FARM, which supports flexibility and scalability.

D. LLM

The LLM portion of our tech stack will be OpenAI, based on the LLM version provided by Virginia Tech. The LLM will convert user requests to vectors, which can deterministically make CRUD requests to the MongoDB database based on various factors. These factors may include game categories, reception, price, and public user data.

V. Checkpoint 1

A. Team Changes

Our project structure has changed because of our original four team members, two have left the team for personal reasons. Going forward, the scope of our project (discussed later) will change along with the delegation of responsibilities. To make up for this change in manpower, the two remaining team members, Steve Park and Christopher Williams will split responsibilities as such:

- **Steve**: Backend
- **Chris**: Frontend

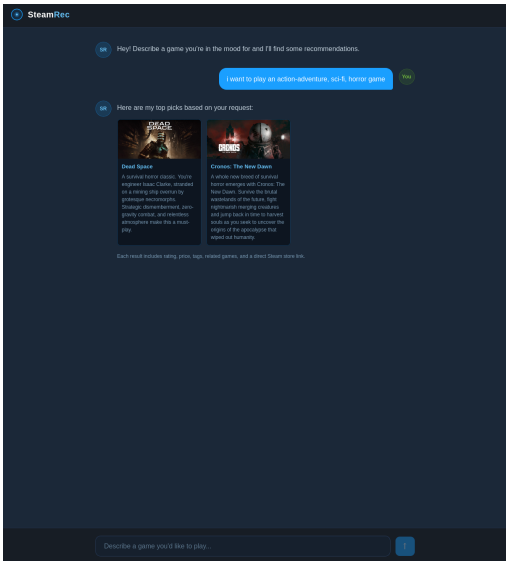

Fig. 3. SteamRec Chat Bot Page

Both team members will collaborate closely and work on the integration of the LLM into the project as well as working on the Database Schema and data ingestion from Steam, and the core structures of the backend and frontend are delegated accordingly.

B. Frontend

A mock-up of the frontend was created to establish an initial direction for the chat window. The goal was a familiar, easy-to-navigate layout modeled after standard chatbot interfaces, with bot responses on the left and user messages on the right. The first iteration was too wide and difficult to read, so the design was revised to a more compact, centered layout.

The frontend is built with React. Currently, the design is being refined while the backend is set up and populated with game data from Steam. Once the backend is stable and contains enough entries, work will shift toward building interactive components that communicate with the database, structure prompts for the LLM, and parse returned queries into readable game recommendations for the user.

Future plans include a landing page, as well as additional pages for the project proposal, checkpoints, team members, project information, and the chatbot itself.

The React frontend, built with Vite and served by nginx in production, communicates with the FastAPI backend through a reverse proxy. Any request to `/api/*` is forwarded by nginx to the backend container, keeping both the SPA and API on the same origin and eliminating the need for CORS configuration. The FastAPI backend will return JSON responses that the frontend parses and displays in the chat window.

Docker Compose manages the dependency chain and internal networking: the frontend depends on the backend, the backend depends on the database, and all three services resolve each other by container name over a shared bridge network. Only the nginx port is exposed to the host, keeping the backend

and database internal. The remaining work is implementing the FastAPI routes and replacing the placeholder data with live API calls.

C. LLM

Our team has decided on using OpenAI for our LLM, utilizing Virginia Tech's free OpenAI API calls. This will allow us to wire in the LLM and make API calls consistently without having to worry about costs and token usage based on price, primarily performance. Accessed from the FastAPI backend via the official OpenAI Python client, when a user submits a query through the chat window, the backend will construct a prompt that includes the user's message along with relevant game data retrieved from MongoDB. This prompt is sent to the OpenAI API, which returns a natural-language response that the backend parses and forwards to the frontend as structured JSON. The API key is stored as an environment variable and injected into the backend container through Docker Compose's `env_file` configuration, keeping credentials out of the codebase. Because all communication with the OpenAI API occurs server-side within the backend container, the key is never exposed to the frontend or the end user. The remaining work involves defining the prompt structure, deciding how much database context to include per request, and handling the response format so the frontend can render game recommendations consistently.

D. Dataset

The dataset for our SteamRec backend consists of about 1,000 unique games from the Steam User API, which is distinguished by the game's appid unique identifier.

1) Data collection: The use of the Steam Web API presents two unique challenges when populating our initial MongoDB server: API calls are limited to 200 every 5 minutes, and the Steam Web API does not directly support RESTful calls to individual games. To solve this issue, we instead use the AppList object to request a json array of all public apps listed on steam's library. We then write a short script to parse through the json array object and manually create readable NoSQL entries per app.

When retrieving our initial dataset, we first make a GET request to Steam Web Service using the following HTTP request:

```
GET https://api.steampowered.com/
        ISteamApps/GetAppList/v2/
```

This returns a JSON object named applist with a JSON array of app objects which we can parse using Python's JSON module. The resulting JSON object looks like the following:

```
{
  "applist": {
    "apps": [
      {"appid": 10, "name":
        "Counter-Strike"},
      {"appid": 20, "name":
        "Team Fortress Classic"},
      {"appid": 30, "name":
        "Day of Defeat"},
      {"appid": 40, "name":
        "Deathmatch Classic"}
    ]
  }
}
```

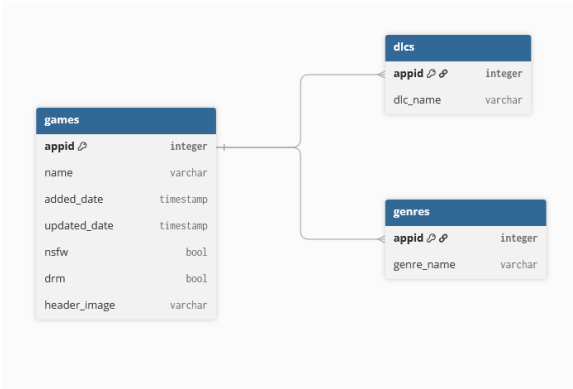

Fig. 4. SteamRec Database Schema

If the user has specific requirements, we can make an additional request for more detail using the appdetails hook:

```
GET https://store.steampowered.com/
            api/appdetails?appids=_
```

To cache our JSON object into NoSQL entries, we iterate through each json object in the applist array and create a new entry defining each column and insert it into our mongodb server, allowing for persistent access to all public Steam games without the limitation on RESTful requests.

VI. Checkpoint 2

A. Frontend

Our frontend received an update and functionality increase. A landing page was created that features an upward scrolling background grid of games, a floating prompt bar in the middle of the page under the splash-hero text, and the logo in the upper-left of the window. The tab now says "SteamRec" and features the SteamRec logo. Upon entering a query, the query will be sent to the LLM and will fetch a response.

A particular difficult that was faced with the design was ensuring that the game images being displayed in the background existed as certain games do not have fetchable header images. In the future, there will be random presentation of game header images that are fetched and ensured that the image exists, but for now there is a fixed, repeating list of game header images that will infinitely loop.

Another minor difficulty was turning the prompt bar into a 'textarea' that would appropriately resize with longer queries instead of the text being hidden and pushed to the left inside the prompt bar.

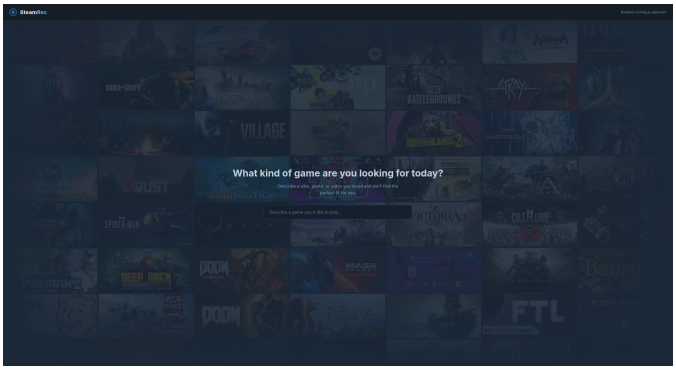

Fig. 5. SteamRec Landing Page

B. LLM

We are using the OpenAI GPT API provided by Virginia Tech, connecting to the ‘gpt-oss-120b‘ model. We currently send user input as the only part of the query with the chat completion POST via the Vite proxy to hide the API key from the browser. We will likely migrate this to the backend so that the entire process is handled server side instead of client side.

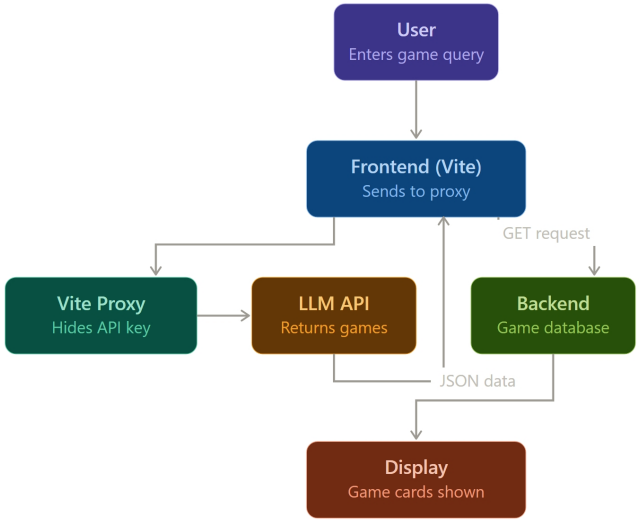

Fig. 6. LLM Querying Process w/ Vite Proxy

The LLM recieves the users query, then POSTs a response that is displayed on the chat page of the LLM. The current query/response back and forth is unstructured, simply recieving the raw ouptut of the model. Our goal is to prompt inject each response with instructions on what to look for, and how to structure its response. Thus, when its structured response is returned, we can resolve this against the database and return games relevant to the query.

Preventing hallucination is a bi-product of the structured response and resolving against the database. When the name of the game in the structured JSON is returned and resolved against the database, it will automatically be unable to return games that do not exist. This also extends to tags and types that do not exist either.

We will explore options to verify veracity and correctness of results with how closely the match the query.

C. Backend

The backend of SteamRec currently serves unprivileged GET requests, which is translated to CRUD requests with fastapi. As our frontend is hosted on port 30, and our backend hosted on port 8000, we chose to include CORS as a simple middleware resource sharing application which allows our frontend to independently host its domain from our backend.

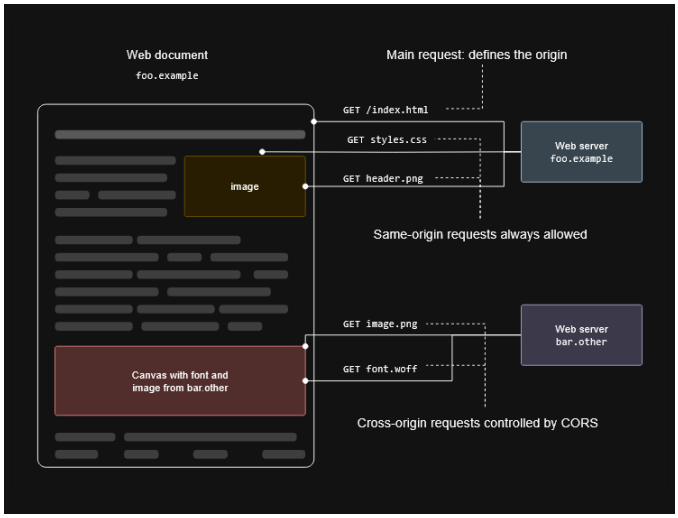

Fig. 7. Model of CORS middleware

Currently our backend serves basic GET requests of apps based on names. For instance, a user may request the json object for the app named ”csgo” by the HTTP request

```
GET http://localhost/apps/csgo
```

which would return a 400 response with the json object as the response body.

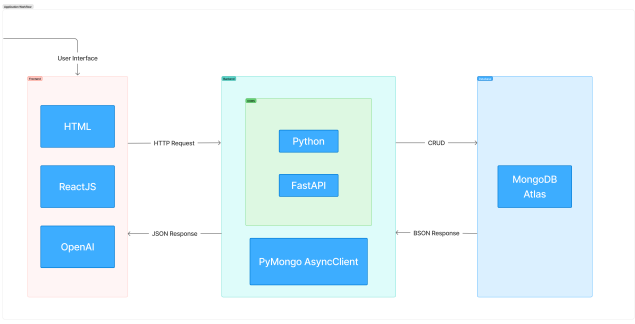

Fig. 8. Application workflow with LLM integration

For readability and scalability, the backend architecture is composed of five directories.

1) config: The config file declares any environment variables we wish to include in our Dockerfile but do not want to push to our public repository. This includes sensitive information such as our Mongo URI and our Steam API key. The config file also determines our host domain name and our backend port number.

2) models: To sanitize dataset records, our backend utilizes pydantic to explicitly declare record classes for apps as well as users. The models directory is essentially an abstract class that is to be instantiated in the routers subdirectory.

3) routers: A naive approach to implementing RESTful methods would be to implement each method in Main. Although this is a practical approach to small-scale projects such as this, another approach would be to fragment RESTful endpoints to subgroups. This layer of abstraction is implemented with FastAPI routers, which group REST methods to one file per subgroup. For SteamRec, we chose to implement two subgroups: apps and user. This allows us to independently investigate the results. contribute to either API implementation and attach them in main to be immediately deployable.

4) schemas: The schemas subdirectory is similar to routers but instead of implementing behaviors of models, it implements behaviors of relationships. The purpose of this subdirectory is to define any HTTP request made that includes a user or app as a component of the request or the response body.

5) main: This is the point which is called by the Dockerfile, loading the asynchronous driver and middleware. All routers are also attached to the async client here.

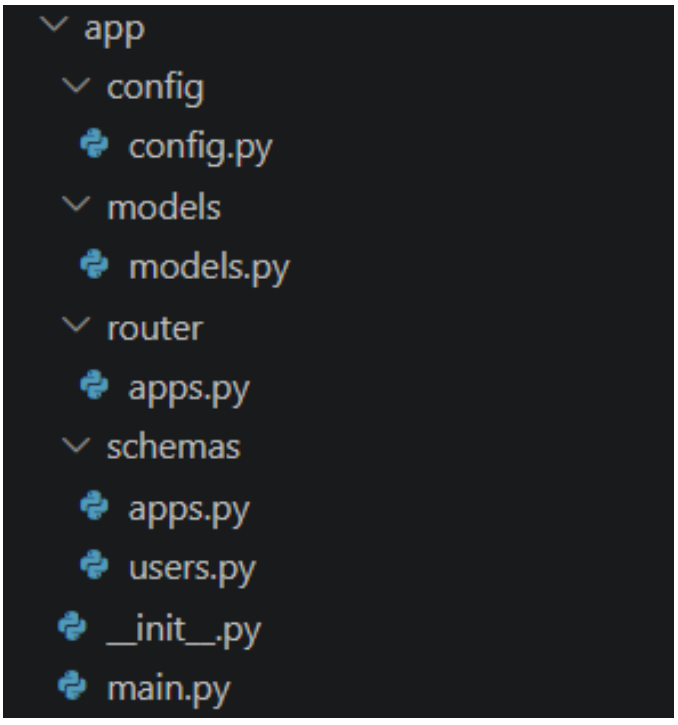

Fig. 9. Backend Architecture of SteamRec

The current working version of SteamRec implements basic GET requests of app names and tags. The goal is to implement GET requests for apps with multiple variables and also integrate OpenID to allow responses taking user profiles into account using session tickets. This would allow for more personalized LLM responses which can take user history into account for select RESTful queries.

REFERENCES

[1] Samin Batra, Vanshika Sharma, Yiwei Sun, Xiao Wang, and Yue Wang. Steam recommendation system. *arXiv preprint*, abs/2305.04890, 2023. Accessed: Mar. 7, 2026.

[2] Robert Blue, Luis Garcia, and Jacob Turner. Game recommendation analysis using Steam profiles and reviews. *SMU Data Science Review*, 8(1), 2024. Accessed: Mar. 7, 2026.

[3] Luke Friedman, Sameer Ahuja, David Allen, Terry Tan, Utkarsh Sidana, Hakim Hamoud, Haris Vikalo, Bharat Rao, Srinivasan Sengamedu, Alex Heitner, and Gaurav Chopra. Leveraging large language models in conversational recommender systems. *arXiv preprint*, abs/2305.07961, 2023. Accessed: Mar. 7, 2026.

[4] Junjie Huang, Wayne Xin Zhao, Hongjin Qian, Congrui Yin, Jing Liu, Zhicheng Dou, Ji-Rong Wen, Eduardo Yaluk, Lixin Chen, and Le Sun. Recommender AI agent: Integrating large language models for interactive recommendations. *ACM Transactions on Information Systems*, 2024. Accessed: Mar. 7, 2026.

[5] Valve Corporation. Introducing the Steam interactive recommender. Steam Blog, March 2020. Accessed: Mar. 4, 2026.

[6] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. A survey on large language models for recommendation. *arXiv preprint*, abs/2305.19860, 2023. Accessed: Mar. 7, 2026.

APPENDIX

TABLE I
PROJECT TEAM MEMBERS AND ROLES

Member	Role
Steve	Backend
Chris	Frontend

- **Week 1:** Project structure and design agreed upon; roles assigned; repository and Docker environment initialized
- **Week 2:** Steam Store API explored; initial data ingestion pipeline implemented; MongoDB schema defined
- **Week 3:** Batch data collection completed; database populated with game metadata including descriptions, tags, prices, and review scores
- **Week 4:** OpenAI API integration implemented; initial prompt design and testing against sample queries
- **Week 5:** Pre-filtering strategy implemented to narrow candidate games before model inference; latency and accuracy evaluated
- **Week 6:** React/TypeScript frontend scaffolded; query input and results display components built
- **Week 7:** Frontend connected to backend API; end-to-end query flow tested and debugged
- **Week 8:** System integration testing; edge cases handled; UI refined
- **Week 9:** Performance evaluation and testing completed; known issues resolved
- **Week 10:** Final review and polish; deployment verified; presentation materials prepared

- **Project Presentation:** Live demonstration of SteamRec; system walkthrough and Q&A
- **Project Report:** Final written report submitted; includes system design, implementation details, evaluation, and reflection